Herzlich willkommen!

Parameter	Kursinformationen
Veranstaltung:	`Erhebung, Analyse und Visualisierung digitaler Daten`
Semester	`Wintersemester 2026/27`
Hochschule:	`Technische Universität Freiberg`
Inhalte:	`Motivation, Lernziele, Arbeitsweise, Semesterplan, Prüfung, Organisation`
Link auf Repository:	https://github.com/TUBAF-IfI-LiaScript/VL_EAVD/blob/master/00_Motivation.md
Autoren	Sebastian Zug & Bernhard Jung & Claude Code (Opus 5.5)

Prof. Dr. Sebastian Zug

Professur für Softwareentwicklung und Robotik

sebastian.zug@informatik.tu-freiberg.de

Prof. Dr. Bernhard Jung

Professur für Virtuelle Realität und Multimedia

bernhard.jung@informatik.tu-freiberg.de

Zum Nacharbeiten: die Ansicht „Lehrbuch“. Im Hörsaal zeigen die Folien nur die Kernbotschaften. Die Erklärungen dazu stehen im Lehrbuch: oben rechts auf das Symbol mit den Schiebereglern klicken und „Lehrbuch“ wählen.

<!– kurslandkarte:knopf-ende ⟶

Warum sind Sie hier?

Was motiviert Sie, an dieser Veranstaltung teilzunehmen?

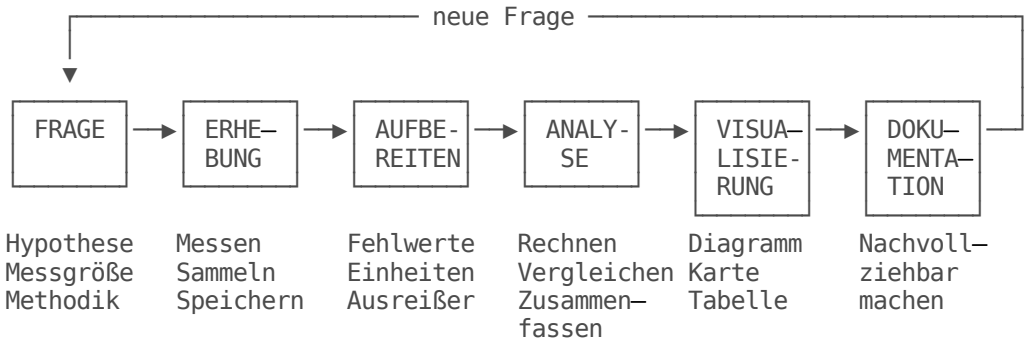
Texteingabe ...

Was verbinden Sie mit dem Begriff „Programmieren“?

Texteingabe ...

Warum ist die Veranstaltung zentral?

> Tip
>
> Die Veranstaltung ist zentral, weil sie die Grundlagen für die Arbeit mit Daten vermittelt. Sie lernen, wie man ausgehend von einer wissenschaftlichen Fragestellung Daten erhebt, aufbereitet, analysiert und visualisiert.



> **Der Pfeil zurück ist der wichtigste.** Fast nie führt der erste Durchlauf zum Ziel — auswerten heißt, diese Schleife mehrfach zu drehen - das bedeutet, man wiederholt immer wieder gleiche Aufgaben.

Von Hand oder mit einem Programm? Diese Frage stellt sich bei jedem Durchlauf.

Die Daten reichen nicht, die Frage war zu unscharf, oder das Ergebnis wirft eine neue Frage auf. Der Name der Veranstaltung nennt drei der Schritte — Erhebung, Analyse und Visualisierung. Ein Programm erledigt sie jedes Mal wieder, nachvollziehbar und ohne Tippfehler; die Tabellenkalkulation verlangt jedes Mal neue Handarbeit.

Lernziele

Am Ende des Semesters können Sie eine wissenschaftliche Fragestellung auf eine Datenverarbeitung abbilden — von der Rohdatei bis zur belegten Aussage. Dafür brauchen Sie beides: das Handwerkszeug und seine Anwendung.

Sie können …	Handwerkszeug (klassische Einführung in die Informatik)	Anwendung auf Daten (eigentliches Ziel)
entwickeln	ein eigenes Programm aus Schleifen, Bedingungen und Funktionen aufbauen	eine vollständige Auswertung schreiben — einlesen, bereinigen, auswerten, darstellen
beurteilen	einschätzen, ob ein Programm verständlich und wartbar geschrieben ist	einschätzen, ob ein Ergebnis belastbar ist und wann eine Darstellung in die Irre führt
analysieren	fremden Python-Code lesen und nachvollziehen, was er tut	einen unbekannten Datensatz erschließen: Was steht drin, was fehlt darin?
anwenden	Variablen, Datentypen, Schleifen und Funktionen zur Lösung einer Aufgabe nutzen	die Werkzeuge der Datenanalyse auf eigene Messreihen und Analysendaten übertragen
verstehen	erklären, was Variable, Datentyp, Schleife, Funktion und Modul bedeuten	erklären, warum Daten aufbereitet werden müssen, bevor man mit ihnen rechnet
erinnern	die Syntax der Grundbausteine benennen	die Grundbegriffe benennen: Algorithmus, Fehlwert, Bibliothek, DataFrame

Die Tabelle liest man am besten von unten nach oben: Erst kennen und verstehen Sie die Grundbausteine, dann wenden Sie sie an, analysieren fremden Code und unbekannte Daten, beurteilen Ergebnisse und entwickeln am Ende eigene Auswertungen. Jede Stufe hat zwei Seiten, das Programmieren selbst und seine Anwendung auf echte Daten. Die Klausur prüft vor allem die mittleren Stufen: Code lesen, verstehen und Fehler darin finden.

Drei Fähigkeiten, die über den Kurs hinaus tragen

1. **Sie erkennen, wann sich Programmieren lohnt** — und wann eine Tabellenkalkulation genügt. Beides zu wissen ist mehr wert, als eine Sprache zu kennen.
2. **Sie können Ergebnisse hinterfragen** — die eigenen und die anderer. Woher kommen die Daten, was fehlt darin, was trägt die Aussage wirklich?
3. **Sie können sich selbst weiterhelfen** — Fehlermeldungen lesen, Dokumentation nutzen, Beispiele übertragen. Kein Kurs deckt ab, was Sie später brauchen werden.

> Caution
>
> **Was ausdrücklich nicht das Ziel ist:** Sie werden keine Software-Entwickler. Kleine Vorhaben können Sie aber selbst umsetzen, größere zumindest einordnen.

Diese drei Fähigkeiten tragen auch dann noch, wenn Sie Python längst gegen ein anderes Werkzeug getauscht haben oder eine KI den Code schreibt: entscheiden, ob sich ein Programm lohnt, einem Ergebnis nicht blind trauen und sich bei einer Fehlermeldung selbst helfen. Und in der Zusammenarbeit mit Fachleuten aus der Informatik können Sie mitreden, weil Sie die Begriffe kennen.

Praktische Inhalte

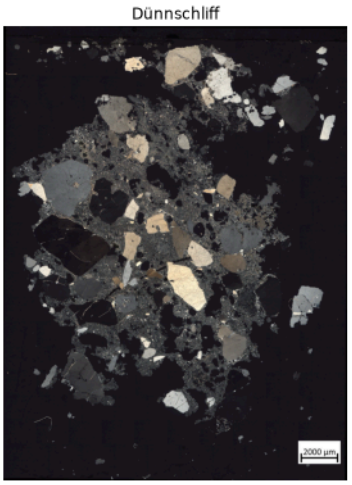
Forschungsfrage: Wie groß sind die Körner in diesem Gestein — und wie viel Fläche nehmen sie ein?

Ein **Dünnschliff** ist eine Gesteinsscheibe, so dünn geschliffen, dass Licht hindurchscheint. Sie müssen heute keine Zeile des Programms verstehen — achten Sie nur auf die Kommentare rechts:

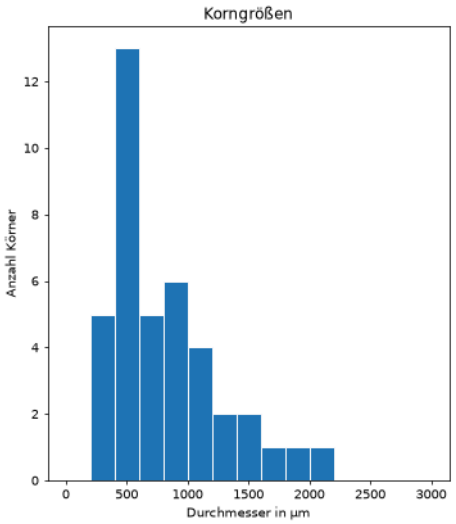
duennschliff.py

```
grau[900:, 640:] = 0            # Maßstab unten rechts ausblenden
UM_PRO_PIXEL = 2000 / 80        # Maßstab: 2000 µm sind 80 Pixel
SCHWELLE = 0.45                 # Annahme: Ein Korn ist heller als die Schwe
hell = ndimage.binary_opening(grau > SCHWELLE, iterations=2)  # Krümel weg
  (Erosion, Dilatation)
koerner, _ = ndimage.label(hell)              # zusammenhängende Flecken
flaeche = np.bincount(koerner.ravel())[1:]    # Pixel je Fleck
flaeche = flaeche[flaeche >= 150]             # sehr kleine Flecken weg
durchmesser = 2 * np.sqrt(flaeche / np.pi) * UM_PRO_PIXEL
```

Ergebnis: **40 Körner**, mittlerer Durchmesser (Median) **657 µm**, zusammen rund **13 %** der Probenfläche. Bild: Fundus von Prof. Heide, TU Bergakademie Freiberg.







Zum Ausprobieren: Das vollständige Programm liegt unter `examples/00_Einfuehrungsbeispiele/Duennschliff/` und läuft in Visual Studio Code.

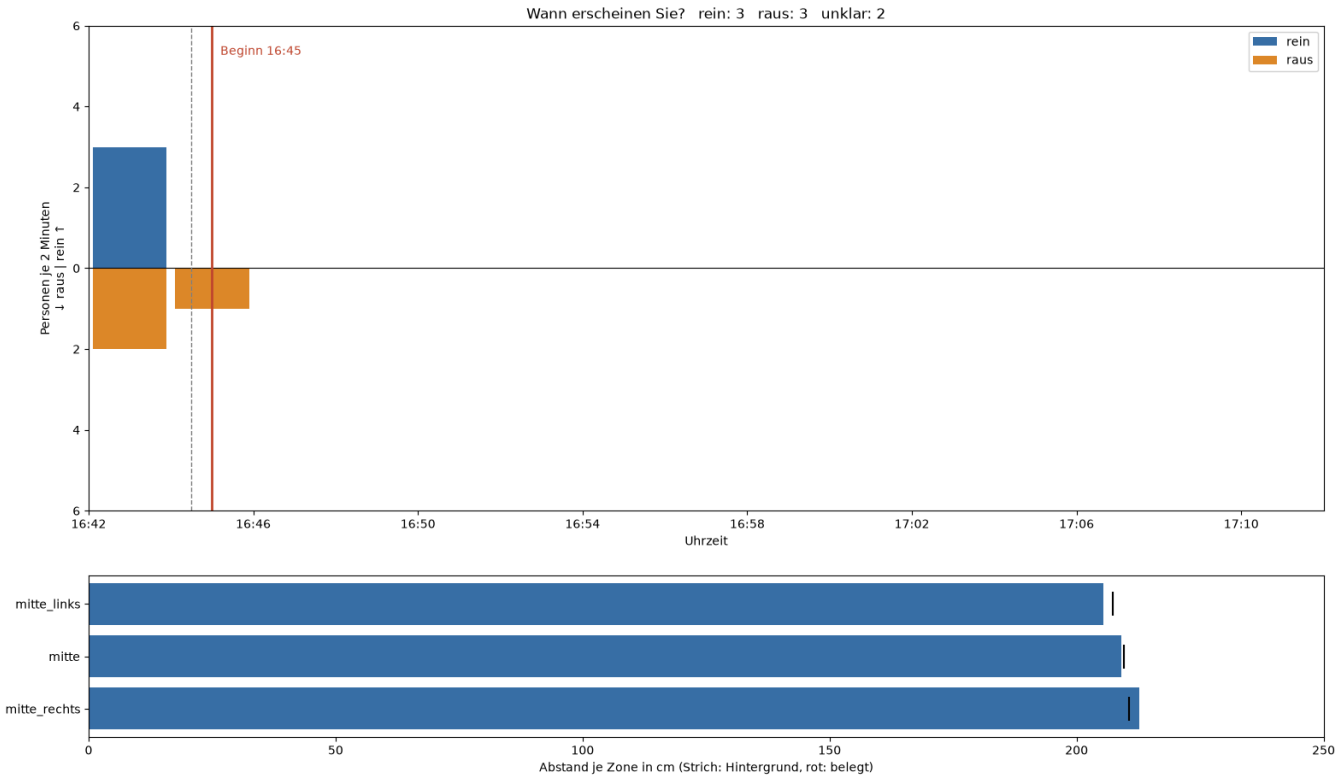
Der Schliff ist etwa 0,03 mm dick. Unter dem Mikroskop liegt er zwischen zwei gekreuzten Polarisationsfiltern; die Mineralkörner erscheinen dann je nach Mineral und Lage hell, farbig oder dunkel. Das Programm sucht die hellen Flecken, zählt sie und rechnet ihre Fläche über den Maßstab in Mikrometer um.

> **Wann erscheinen Studierende in der Vorlesung?** Gibt es Muster — der frühe Vogel, rechtzeitig aber knapp, in den ersten zehn Minuten passiert ohnehin nichts?

Ein kleiner Roboter an der Hörsaaltür zählt, wer hereinkommt und wer hinausgeht — auch heute, während Sie hereinkommen. Ein Auszug aus einem Probelauf:

zaehlungen_2026-10-15.csv

```
zeit;richtung
2026-10-15T16:42:31;raus
2026-10-15T16:42:32;unklar
2026-10-15T16:42:36;raus
2026-10-15T16:42:40;rein
2026-10-15T16:43:04;unklar
2026-10-15T16:43:08;rein
```



Eine einfache Frage, echte Messdaten — und schon stecken wir mitten in den Themen dieses Semesters. Am Ende dieser Vorlesung kommen wir darauf zurück.

Und was ist mit Ihnen?

> **Tip**
>
> Welche Forschungsfragen sind für Sie interessant?

Aus welchem Studiengang kommen Sie?

- ◯ GING
- ◯ SR
- ◯ US
- ◯ WIW
- ◯ GM
- ◯ NT
- ◯ BAF
- ◯ ENG
- ◯ weitere

Welche Daten würden Sie gern analysieren?

Texteingabe ...

Welche Programmiererfahrungen haben Sie?

- ◯ Keine, ich habe noch nie programmiert.
- ◯ Ein bisschen, ich habe schon mal ein Programm geschrieben.
- ◯ Viel, ich habe schon mehrere Programme geschrieben.

Kernkonzepte der Vorlesung

Jede Vorlesung beginnt mit einer Frage an echte Daten. Die Sprachkonzepte kommen dann, wenn man sie zur Beantwortung braucht.

```
statt:    "Heute beschäftigen wir uns mit Schleifen."
sondern:  "Wir müssen 48.000 Zeilen durchgehen. Wie machen wir das?"
                                    ↓
                          Dafür gibt es Schleifen.
```

Wer weiß, *wozu* ein Konzept dient,

- … behält es.
- … für denjenigen ist die Programmiersprache irrelevant.
- … kann besser mit der KI zusammenarbeiten.

Wer ein Konzept als Vokabel lernt, vergisst es nach der Klausur. Um einige Grundlagen kommen wir trotzdem nicht herum — aber immer mit Blick darauf, wofür wir sie brauchen.

Ein Datensatz durch das ganze Semester

Roter Faden ist die Klimamessreihe des Deutschen Wetterdienstes vom **Fichtelberg** (1213 m): Tageswerte seit 1890, fast 48.000 Zeilen, frei verfügbar — und mit allen Widrigkeiten echter Daten.

produkt_klima_tag_18900801_20251231_01358.txt

```
STATIONS_ID;MESS_DATUM;QN_3;  FX;  FM;QN_4; RSK;RSKF; SDK;SHK_TAG;  NM; VPM
  TMK; UPM; TXK; TNK; TGK;eor
        1358;19101210;-999;-999;-999;    1;   0.0;   0;-999;  90;   3.3;   7
          863.60;    3.8;   90.00;    5.4;    1.0;-999;eor
        1358;19151001;-999;-999;-999;    1;   2.0;   1;-999;   0;   8.0;   7
          873.70;    2.0;  100.00;    2.8;    1.0;-999;eor
```

Fehlwerte als `-999`, kryptische Spaltennamen, Datumsangaben ohne Trennzeichen — und zwischen diesen beiden Zeilen fehlen fast fünf Jahre.

Dass dieser Datensatz „schmutzig“ ist, ist kein Nachteil, sondern der Punkt. An sauberen Datensätzen lernt man nichts über Datenanalyse.

Dazu kommen Beispiele aus Ihren Fächern — Analysendaten, Bohrprofile, Pegelstände, Rohstoffpreise.

Ihre Antworten auf die Umfrage „Und was ist mit Ihnen“ helfen uns bei der Auswahl.

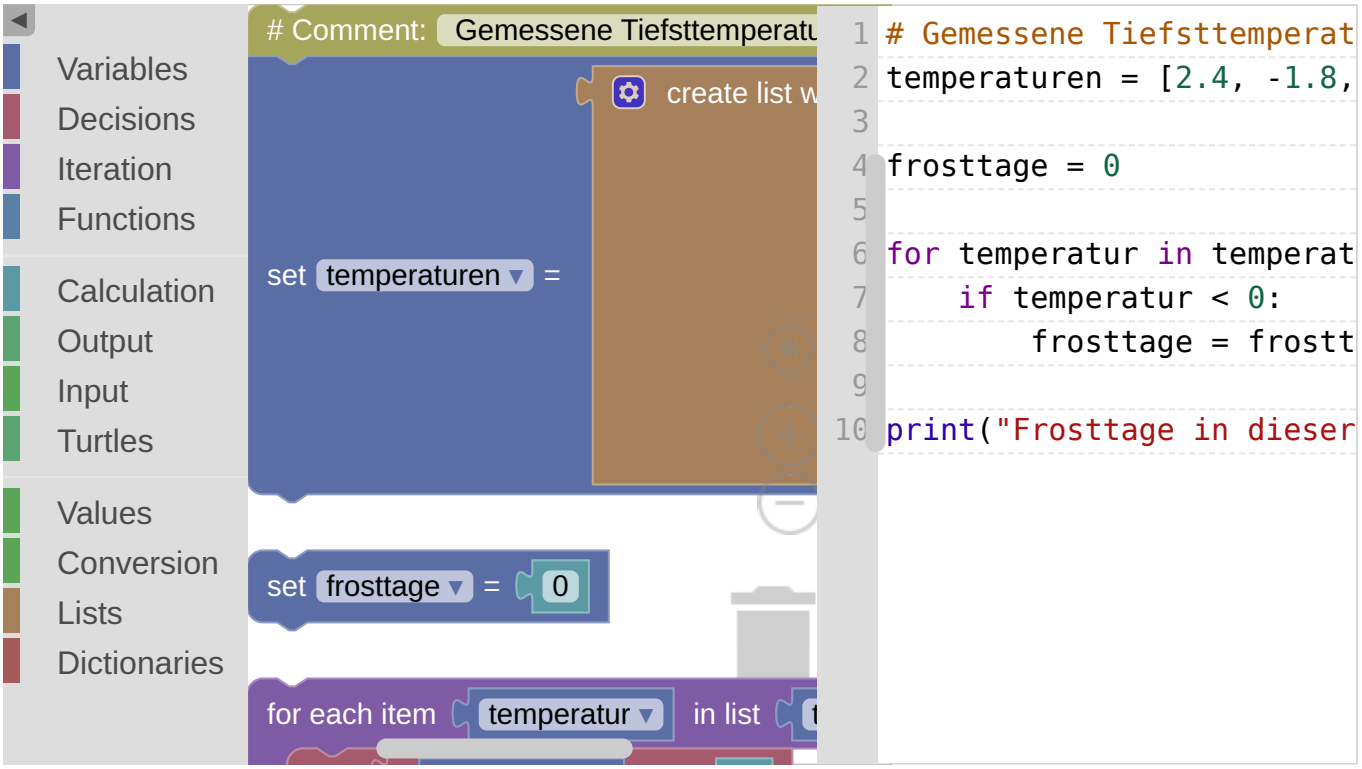
Blöcke oder Text

Caution

Das Laden der Beispiele dauert im Browser ein paar Sekunden. Bitte warten Sie, bis der Editor erscheint.

Die ersten Beispiele können Sie wahlweise als **Blöcke** zusammenschieben oder als **Text** tippen. Das sind nicht zwei Sprachen, sondern zwei Ansichten auf dasselbe Programm.

Eine Woche mit Tiefsttemperaturen — gezählt werden die **Frosttage**, an denen das Minimum unter 0 °C lag. Legen Sie sich fest, **bevor** Sie das Programm ausführen: Wie viele sind es?



Nutzen Sie die Schaltflächen über dem Editor, um zwischen Blöcken und Text zu wechseln. In Vorlesung 03 verstehen Sie jede dieser Zeilen.

Typische Fehlvorstellungen

In jeder Vorlesung begegnen Ihnen Annahmen, die zu Programmen führen, die ohne Fehlermeldung falsch rechnen. Sie geben vorher eine Vorhersage ab — und merken, wenn Sie danebenliegen. Ein Beispiel:

> **Typische Fehlvorstellung:** *„Wenn das Programm ohne Fehlermeldung durchläuft, stimmt das Ergebnis.“*

Vier Tagesmittel vom Fichtelberg, eines davon fehlt und steht als `-999` in der Datei. Legen Sie sich fest, **bevor** Sie das Programm ausführen: Was gibt es aus?

mittel.py

```
1  temperaturen = [-1.8, -3.5, -999, 0.7]
2  mittel = sum(temperaturen) / len(temperaturen)
3  print("Mittlere Temperatur:", round(mittel, 1))
```

```
Another process is running, wait until finished
```

> **Tip**
>
> Fehlvorstellungen sind ein typisches Problem, das fast alle am Anfang haben. Wer sie kennt und versteht, läuft nicht in die Falle.

Python rechnet `-999` klaglos mit: (-1,8 - 3,5 - 999 + 0,7) / 4 = -250,9. Keine Fehlermeldung, aber ein unsinniges Ergebnis. Ohne den Fehlwert wären es -1,5 °C. Wie man Fehlwerte erkennt und auslässt, lernen Sie in Vorlesung 03.

Erst mühsam, dann mit Werkzeug

> **Die Wetterdatei einlesen: in Vorlesung 06 rund 50 Zeilen, in Vorlesung 11 eine Anweisung.**

Dieselbe Frage zweimal — wie warm ist es auf dem Fichtelberg im Mittel aller Tage seit 1890? Von Hand (Vorlesung 06, hier nur der Kern):

von_hand.py

```
werte = []
dateiname = "data/fichtelberg/produkt_klima_tag_18900801_20251231_01358.txt
with open(dateiname) as datei:
    datei.readline()                    # Kopfzeile überspringen
    for zeile in datei:
        spalten = zeile.split(";")
        tmk = float(spalten[13])        # Spalte 13: Tagesmittel TMK
        if tmk != -999:                 # Fehlwert auslassen
            werte.append(tmk)
print(round(sum(werte) / len(werte), 2))
```

Mit der Bibliothek pandas (Vorlesung 11):

mit_pandas.py

```
import pandas as pd
dateiname = "data/fichtelberg/produkt_klima_tag_18900801_20251231_01358.txt
df = pd.read_csv(dateiname, sep=";", skipinitialspace=True, na_values=-999)
print(round(df["TMK"].mean(), 2))
```

Beide liefern `3.24` (°C). Wer den mühsamen Weg gegangen ist, versteht, *was* `read_csv` abnimmt — Trennzeichen, Kopfzeile, Fehlwerte. Wer ihn übersprungen hat, steht hilflos da, sobald es einmal nicht funktioniert.

Und was ist mit KI?

> **Wer die Begriffe nicht kennt, kann auch die Hilfe einer KI nicht nutzen.**

KI-Assistenten schreiben in Sekunden Python-Code. Aber Sie müssen sagen können, was Sie wollen — und prüfen können, ob das Ergebnis stimmt.

	Ohne Begriffe	**Mit Begriffen**
Frage an die KI	„Wie warm ist es auf dem Fichtelberg?“	„Lies die DWD-Datei mit `;` als Trennzeichen, lass in der Spalte TMK die Fehlwerte `-999` weg und bilde den Mittelwert.“
Antwort prüfen	„Das Programm läuft, also stimmt es.“	„Wird `-999` wirklich ausgelassen? Ist es die richtige Spalte?“

Eine KI ist ein Werkzeug wie die Tabellenkalkulation oder pandas: Sie nimmt Arbeit ab, aber kein Verständnis. Erinnern Sie sich an die -250,9 °C: Ein Programm von einer KI, das den Fehlwert mitrechnet, läuft genauso ohne Fehlermeldung. Wer Code lesen kann und die Begriffe kennt, merkt das. Genau das üben wir in diesem Semester — und genau das prüft die Klausur.

Vorlesungsmaterialien

In der Vorlesung	Werkzeug	Wofür
in jeder Vorlesung	Beispiele direkt im Browser	ausprobieren ohne Installation
ab Vorlesung 03	dazu Python in Visual Studio Code	Programme mit echten Dateien
ab Vorlesung 10	dazu Jupyter Notebooks in Visual Studio Code	schrittweise Datenanalyse mit Diagrammen

In den Übungen arbeiten Sie mit Visual Studio Code auf Ihrem eigenen Rechner — eingerichtet wird es in der ersten Übung.

> Die Beispiele im Browser sind gewöhnliches Python. Sie lassen sich eins zu eins auf Ihren Rechner übertragen.

Dasselbe Mini-Programm hier im Browser und gleich in Visual Studio Code als Datei `welcome.py`:

welcome.py

```
1 print("Herzlich willkommen in der Vorlesung!")
```

```
Herzlich willkommen in der Vorlesung!
```

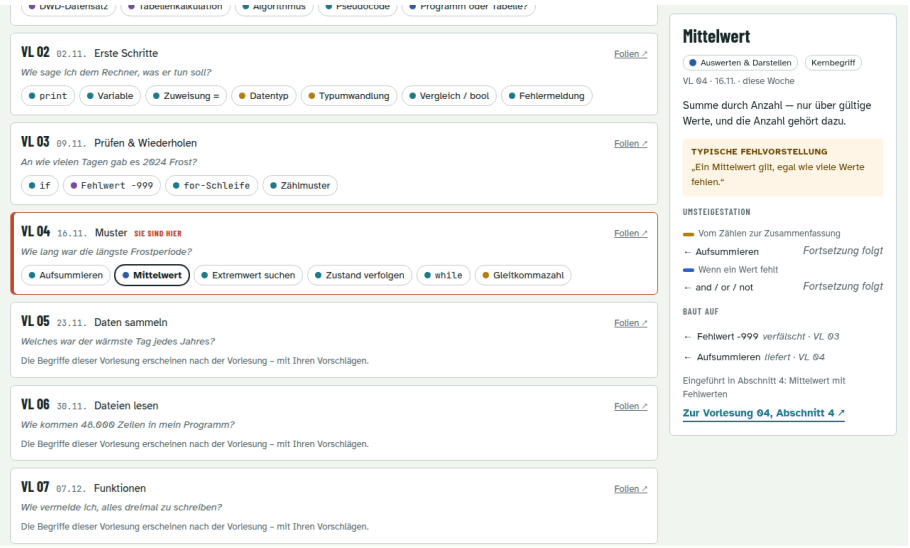
Beide Male lautet die Ausgabe `Herzlich willkommen in der Vorlesung!`. Nur der Ort, an dem Python läuft, ist ein anderer. Die Datei liegt auch im Repository: `examples/00_Einfuehrungsbeispiele/welcome.py`.

Die Kurslandkarte

> **Wo stehen wir — und worauf baut auf, was heute dazukommt?**

Die Kurslandkarte zeigt für jede Vorlesung die neuen Begriffe, und zu jedem Begriff

- eine Erklärung in einem Satz,
- die typische Fehlvorstellung, mit der viele am Anfang danebenliegen,
- worauf er aufbaut und wohin er führt,
- einen Link direkt zur passenden Folie.



Die Karte wächst mit dem Semester — nach jeder Vorlesung kommen ihre Begriffe dazu, auch Ihre Vorschläge.

Programmieren lernt man in Schichten: Jeder neue Begriff stützt sich auf frühere. Die Karte hält diesen Zusammenhang fest. Für kommende Vorlesungen stehen nur Termin und Leitfrage — ihre Begriffe entstehen erst nach der Vorlesung, mit Ihren Vorschlägen. „Sie sind hier" richtet sich nach dem Datum. Neben der Liste gibt es eine Metrokarte, deren Linien rote Fäden durch das Semester sind — etwa „Wenn ein Wert fehlt" —, und eine Zeitleiste mit allen Begriffen nach Themengebiet.

> **Note**
>
> Am Ende jeder Vorlesung fragen wir Sie nach den Stichworten des Tages und tragen sie in die Kurslandkarte ein.

Offene Materialien

- **Im Browser, ohne Installation:** geschrieben mit LiaScript, Beispiele zum Ausführen und Verändern.
- **Präsentation im Hörsaal, Lehrbuch zum Nacharbeiten:** Umschalten über das Symbol mit den Schiebereglern oben rechts.
- **Frei verfügbar:** Lizenz CC BY 4.0, offen auf GitHub.
- **Lebendig:** Die Versionsnummer im Kopf jeder Vorlesung zeigt, wann sich etwas geändert hat.
- **Machen Sie mit:** Fehler melden Sie im OPAL-Forum oder per E-Mail. Wer zu einer Vorlesung beiträgt, wird dort als **Mitautorin oder Mitautor** genannt.

https://github.com/TUBAF-IfI-LiaScript/VL_EAVD

Die dritte Ansicht, „Folien“, zeigt die Erklärungen als Notiz unter jeder Folie. Die Materialien dürfen Sie nutzen, teilen und bearbeiten, solange Sie die Urheber nennen. Wer GitHub nutzt, kann Fehler auch als Issue melden oder Verbesserungen direkt als Änderung vorschlagen.

Organisation der Veranstaltung

Die Vorlesung findet montags von 16:15 bis 17:45 Uhr im Raum FOR-0270 statt. Vor Weihnachten legen wir die Grundlagen, danach geht es um Datenanalyse.

Termin	VL	Thema	Leitfrage
19.10.	00	Motivation, Organisation	Warum sollte ich programmieren lernen?
26.10.	01	Vom Problem zum Programm	Wo hört die Tabellenkalkulation auf?
02.11.	02	Erste Schritte in Python	Wie sage ich dem Rechner, was er tun soll?
09.11.	03	Prüfen und Wiederholen	An wie vielen Tagen gab es 2024 Frost?
16.11.	04	Muster in Messreihen	Wie lang war die längste Frostperiode?
23.11.	05	Daten sammeln	Welches war der wärmste Tag jedes Jahres?
30.11.	06	Dateien lesen	Wie kommen 48.000 Zeilen in mein Programm?
07.12.	07	Funktionen	Wie vermeide ich, alles dreimal zu schreiben?
14.12.	08	Demonstration: Datenerhebung mit MicroPython	Wo kommen die Daten eigentlich her?
04.01.	09	Programme strukturieren	Wie organisiere ich Code, der wächst?
11.01.	10	Bibliotheken: NumPy	Warum muss ich das Rad nicht neu erfinden?

18.01.	11	pandas I — Einlesen	48.000 Zeilen in einer Anweisung?
25.01.	12	pandas II — Aggregieren	Wie fasse ich Jahrzehnte zusammen?
01.02.	13	Visualisierung	Wie zeige ich, was ich gefunden habe?
08.02.	14	Datenqualität, Ausblick, Prüfungsvorbereitung	Kann ich meinem Ergebnis trauen?

Prüfung

Die Veranstaltung schließt mit einer **schriftlichen Prüfung** ab.

Eine Klausur prüft nicht, ob Sie eine Analyse *bauen* können, sondern ob Sie Code **lesen, verstehen, korrigieren und skizzieren** können. Darauf bereiten wir gezielt vor:

Kompetenz	**Wie sie geübt wird**
Code lesen und Ergebnis vorhersagen	Vorhersage-Fragen in jeder Vorlesung
Fehler finden	Live Hacking mit echten Irrwegen, typische Fehlvorstellungen
Code auf Papier schreiben	Übungsaufgaben ohne Rechner
Konzepte erklären	Skizzenaufgaben, Schwerpunkt Datenanalyse

Was in der Klausur vorkommt

- *Welchen Wert gibt das folgende Programm in Zeile x aus?*
- *Finden Sie alle syntaktischen und logischen Fehler im nachfolgenden Code.*
- *Schreiben Sie eine Funktion, die …*
- *Was macht* `groupby` *mit dieser Tabelle? Skizzieren Sie das Ergebnis.*
- *Warum ist diese Darstellung irreführend?*
- *Warum ist* `-999` *in diesem Datensatz ein Problem?*

Was nicht vorkommt: auswendig gelernte Bibliothekssyntax und die Inhalte der Mikrocontroller-Demonstration.

Vorlesung 14 enthält eine Prüfungsvorbereitung mit Musteraufgaben.

> Studierende der *Einführung in die Informatik* (7 LP) bearbeiten neben der Klausur eine praktische Programmieraufgabe. Thema und Umfang stimmen Sie mit den Übungsbetreuern ab.

Übungen

Übungen. Die Übungen vertiefen jede Vorlesung an Programmieraufgaben zum Datensatz der Woche — ein Teil davon bewusst auf Papier, als Vorbereitung auf die Klausur. Sie beginnen in der zweiten Vorlesungswoche, also ab Montag, 26.10. Wählen Sie einen der vier Termine:

Tag	Uhrzeit	Raum
Montag	14:30–16:00	RAM-2119
Mittwoch	8:00–9:30	RAM-2119
Mittwoch	8:00–9:30	KKB-2097
Mittwoch	18:00–19:30	RAM-2119

In der ersten Übung richten wir gemeinsam Python und Visual Studio Code ein. Danach gehört jedes Übungsblatt zur Vorlesung der Woche: Sie versuchen sich bis zur Übung der Folgewoche daran, dort besprechen wir es.

Die Übungsblätter finden Sie jede Woche im OPAL-Kurs der Veranstaltung, nach der Vorlesung. Melden Sie sich dafür im OPAL-Kurs an und tragen Sie sich dort in eine der Übungsgruppen ein. Wie ein Blatt aussieht, zeigt Blatt 1 — die Aufgaben zu Vorlesung 01 und die Einrichtung von Python für die erste Übung.

Materialien. Alle Vorlesungen, Daten und Beispiele finden Sie unter github.com/TUBAF-IfI-LiaScript/VL_EAVD; wie Sie mitmachen können, steht im Abschnitt „Offene Materialien“.

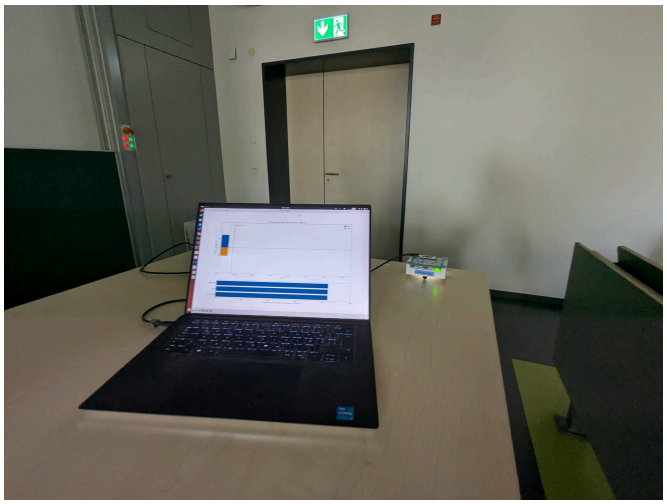
Zeitaufwand. Für die Vorlesung erhalten Sie 6 / 7 CP. Das ist gleichbedeutend mit 180 h / 210 h: 60 h Präsenzzeit und 120 h Selbststudium für Vor- und Nachbereitung, Übungsaufgaben und Prüfungsvorbereitung. Ohne Programmiererfahrung sollten Sie eher mehr Zeit einplanen — in einer Arbeitsgruppe macht das deutlich mehr Spaß als allein.

Wie Sie zum Gelingen beitragen können

- Stellen Sie Fragen — in der Vorlesung, in der Übung, im OPAL-Forum.
- Bringen Sie Beispiele aus Ihrem Fach mit.
- Melden Sie Fehler und Verbesserungsvorschläge zu den Materialien — im OPAL-Forum, per E-Mail oder als Issue auf GitHub.

Wo soll das hinführen?

Schauen wir, um diese Motivation abzurunden, noch einmal auf unser Beispiel mit dem Personenzähler — es zeigt die als Einleitung aufgeführte Verarbeitungskette aus Datenerhebung, Analyse und Visualisierung.



Auf dem Roboter läuft ein kleines Python-Programm (MicroPython, vereinfacht). Es misst immer wieder fünf Abstände nach vorn und gibt sie als Textzeile aus:

alvik_messung.py

```
alvik = ArduinoAlvik()
alvik.begin()

while True:
    werte = alvik.get_distance("mm")
    zahlen = [str(int(w)) for w in
      werte]
    print(ticks_ms(), ";".join(zahle
      ), sep=";")
    sleep_ms(50)
```

https://docs.arduino.cc/tutorials/alvik/user-manual/#distance-sensor

https://docs.arduino.cc/tutorials/alvik/api-overview/#get_distance

Über das USB-Kabel kommt am Laptop an — jemand geht von links nach rechts vorbei:

Ausgabe des Roboters (Probelauf)

```
ms;links;mitte_links;mitte
  ;mitte_rechts;rechts
53566;2393;2101;2106;2111;2239
53670;797;2092;2101;2110;2219
53876;766;778;1437;2097;2194
```

Daraus macht das Programm auf dem Laptop Zählungen

zaehlungen_2026-10-15.csv

```
zeit;richtung
2026-10-15T16:29:15;rein
2026-10-15T16:29:18;raus
2026-10-15T16:29:21;rein
```

Welche kritischen Fragen müssen wir uns stellen?

- Wie kommen die Daten zustande? Was sind Annahmen, die möglicherweise an anderer Stelle nicht gelten (Übertragbarkeit der Lösung)?
- Was passiert im Fehlerfall - wenn zum Beispiel das Kabel abgezogen wird?
- Welche Daten speichern wir überhaupt? Was ist ein geeignetes Format?
- Wie können wir die Daten visualisieren, um sie zu verstehen? Welche Darstellungsform ist geeignet?

> **Tip**
>
> Wer von Ihnen gleich im Anschluss an die Vorlesung noch ein wenig mit dem Roboter spielen möchte, ist herzlich eingeladen. Das vollständige Projekt — Programm für den Roboter, Zähler mit Live-Anzeige und Anleitung — liegt unter `examples/00_Einfuehrungsbeispiele/Alvik/`.

Zu Beginn lernt der Roboter, wie weit die Wand gegenüber entfernt ist. Im Probelauf hat er zwei Durchgänge nicht einordnen können und als „unklar“ notiert. In Vorlesung 08 sehen wir eine solche Messung live auf einem Mikrocontroller.

Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?

Texteingabe …

Was blieb unklar?

Texteingabe …

Welche Anwendungsfälle sind denkbar?

Texteingabe …

Die letzte Frage zielt darauf, dass ich gezielt Beispiele aus Ihrem Fachgebiet in die Vorlesung einbauen möchte. Dazu spreche ich dann die Kollegen der Fachbereiche an. Wer möchte, kann mir auch selbst Beispiele zuschicken.

Nächste Woche

Sie bekommen die Datei des Personenzählers und die Fichtelberg-Daten — und versuchen, drei Fragen mit einer Tabellenkalkulation zu beantworten. Ohne eine Zeile Code.

Bringen Sie einen Laptop mit einer Tabellenkalkulation mit — LibreOffice Calc, Excel oder Google Sheets.